University of Zagreb
Faculty of Mechanical Engineering and Naval Architecture

BACHELOR'S THESIS

Andrija Adamović

Zagreb, 2026

University of Zagreb

Faculty of Mechanical Engineering and Naval Architecture

BACHELOR'S THESIS

Mentor:

Prof. dr. sc. Andrej Jokić

Student:

Andrija Adamović

Zagreb, 2026

I hereby declare that I have written this thesis independently, using the knowledge acquired during my studies and the literature cited in the references.

TODO: Zahvale

Andrija Adamović

SVEUČILIŠTE U ZAGREBU
FAKULTET STROJARSTVA I BRODOGRADNJE

Središnje povjerenstvo za završne i diplomske ispite
Povjerenstvo za završne ispite studija mehatronika i robotika

Sveučilište u Zagrebu Fakultet strojarstva i brodogradnje	
Datum	Prilog
Klasa: 602-01/26-01/3	
Ur.broj: 15-31000-26-	

ZAVRŠNI ZADATAK

Student: **Andrija Adamović** JMBAG: 0035248277

Naslov rada na hrvatskom jeziku: **Willemsova fundamentalna lema s primjenama**

Naslov rada na engleskom jeziku: **Willems' Fundamental Lemma with Applications**

Opis zadatka:

Classical approach to simulation and controller design for dynamical systems is a model-based approach. A model of system's dynamics is obtained either directly by applying laws of physics on the considered system or indirectly via system identification. The latter is a data-based approach and the data, which consists of recorded input-output trajectories of the system, is used in a suitably defined optimization problem to obtain a model.
Alternative to model-based approaches is the direct data-based approach where simulations and/or controller synthesis is performed directly from data, completely avoiding a model. Such approach has gained significant attention in the control systems community over the past years. There are several reasons for that and the obvious one is that modern IT technologies enable collecting (sensing/recording) of large data sets in an inexpensive way. From a scientific point of view, the direct data-based approach is attractive as it can offer exact results with robustness guarantees, similar to the classical model-based approach.
One of the key results that has inspired and enabled a set of methods in direct data-based approach is the so-called Willems' fundamental lemma. Loosely speaking, this lemma states that all possible trajectories of a linear time invariant (LTI) dynamical system belong to an image space of a suitable defined data-based matrix.
The main goal of this thesis is to summarize dominant state-of-the-art approaches to data-based simulation and controller design and to suitably illustrate them on several examples.
The thesis should contain the following:

- Presentation of the Willems' fundamental lemma together with discussion of its origins and meaning.
- Literature overview of the state-of-the-art results on data-based approaches which are based on the Fundamental lemma.
- On a set of suitably selected and designed examples illustrate some of the data-based approaches for simulation and control of LTI dynamical systems.
- Using suitable simulation tools (e.g., MATLAB/Simulink) to investigate applicability of the Fundamental lemma on nonlinear systems.

The thesis must contain the list of used literature as well as any assistance received.

Zadatak zadan:
22. 4. 2026.

Datum predaje rada:
2. rok: 15. i 16. 7. 2026.
3. rok: 16. i 17. 9. 2026.

Predviđeni datumi obrane:
2. rok: 21. - 23. 7. 2026.
3. rok: 22. - 24. 9. 2026.

Zadatak zadao:
Prof. dr. sc. Andrej Jokić

Predsjednik Povjerenstva:
izv. prof. dr. sc. Petar Ćurković

Contents

List of Figures

List of Tables

LIST OF SYMBOLS

Symbol	Description	Unit
x	Example variable	/

LIST OF ABBREVIATIONS

Abbreviation	Description
LTI	Linear time-invariant

SAŽETAK

Ovdje upišite sažetak rada na hrvatskom jeziku.

KLJUČNE RIJEČI: ključna riječ jedan; ključna riječ dva

ABSTRACT

Write the thesis abstract in English here.

Keywords: keyword one; keyword two; keyword three

1 Introduction

Classical control systems almost always use a system model to synthesise a control algorithm.

1.1 Motivation

Describe the background and motivation for the thesis.

1.2 Literature Overview

2 The Fundamental Lemma

2.1 Theoretical Background

The goal of this section is to provide introduction to mathematical concepts needed for understanding the rest of the thesis. We will lay out some famous results from linear control systems theory.

2.1.1 LTI Dynamical Systems

Although Willems' Fundamental Lemma stems from the behavioural approach to sytems theory, in this thesis it is formulated using the classical state-space LTI system formulation. This formulation is simpler, yet sufficient for the further chapters of the thesis.

Definition 1. *A discrete-time linear time-invariant (LTI) system is a dynamical system whose dynamics are described by:*

$$x(t+1) = Ax(t) + Bu(t) \tag{2.1a}$$
$$y(t) = Cx(t) + Du(t) \tag{2.1b}$$

Where $t \in \mathbb{Z}_{\geq 0}$ is the time axis, $x(t) \in \mathbb{R}^n$ is the state, $u(t) \in \mathbb{R}^m$ is the input and $y(t) \in \mathbb{R}^p$ is the output of the system.
The true state space dimension is n and system matrices $A \in M_n(\mathbb{R})$, $B \in M_{n\times m}(\mathbb{R})$, $C \in M_{p\times n}(\mathbb{R})$, $D \in M_{p\times m}(\mathbb{R})$.

In our data-based approach, those matrices (i.e. the system model) are generally unknown. However, an upper bound on state space dimension is given $N \geq n$. The concept of a system trajectory is defined now.

Definition 2. *A sequence of ordered triples $(u(t), x(t), y(t))_{t=0}^{\infty}$ is called an input-state-output trajectory if*

$$\begin{bmatrix} x(t+1) \\ y(t) \end{bmatrix} = \begin{bmatrix} A & B \\ C & D \end{bmatrix} \begin{bmatrix} x(t) \\ u(t) \end{bmatrix}$$

We can be deduce that, from the linearity of (2.1), any linear combination of input-state-ouput trajectories is itself an input-state-output trajectory i.e. the set of all such trajectories spans a vector space over $\mathbb{R}$. That space is called the *behaviour* of the system. Additionally, from time invariance (2.1) it can be deduced that:

$$(u(t+\tau), x(t+\tau), y(t+\tau))_{t=0}^{\infty}$$

is an input-state-output trajectory for all $\tau \in \mathbb{N}$

Definition 3. *A sequence of ordered pairs $(u(t), y(t))_{t=0}^{\infty}$ is called an input-output trajectory if there exists a sequence of states $x : \mathbb{Z}_{\geq 0} \to \mathbb{R}^n$ such that $(u(t), x(t), y(t))_{t=0}^{\infty}$ is an input-state-output trajectory.*

In practice we are limited by finite computer memory so it is not possible work with infinite sequences of data like the trajectories defined above, so we need to take a snippet of the trajectory relevant to us. Motivated by that we state the next definition.

Definition 4. *Let $i, j \in \mathbb{Z}_{\geq 0}$ such that $i \leq j$. Given an input-state-output trajectory $(u(t), x(t), y(t))_{t=0}^{\infty}$, the sequence $(u(t), x(t), y(t))_{t=i}^{j}$ is called a restricted input-state-output trajectory. Restricted input-output trajectories are defined in the same way.*

2.1.2 The Concepts of Contollability and Observability

In this section the concepts of controllability and observability will be defined. Since, we defined the LTI system (2.1) classically the same will follow for the next definitions. However, although the classical and behavioral notions of controllability and observability are closely related, they are not identified unconditionally. Their equivalence depends on how the behavior is represented. In particular, when considering input-output behavior, equivalence with the classical state-space definitions typically requires a minimal realization; for the full input-state-output behavior, the correspondence is more direct. We will not bother with those comparisons between approaches, more on that topic can be found at (TODO: link za clanak/knjigu).

Definition 5. *A discrete LTI system (2.1) is called controllable if for any initial state $x(0)$ and any desired state x_f, there exists a finite-length sequence of inputs*

$$u(0), u(1), ..., u(N-1)$$

such that the system reaches $x(N) = x_f$. Otherwise, the system is said to be uncontrollable.

In lays terms, it is telling that any state of the system can be reached in finite time by applying finitely many inputs.
We will now state the famous result for testing system controllability stated and proven by Rudolf. E. Kalman. The pair (A, B) is the pair of matrices from (2.1) and n is the true state space dimension of the system.

Theorem 1. *(Kalman controllability theorem) The pair (A, B) is controllable if and only if the controllability block matrix*

$$\mathcal{C} = \begin{bmatrix} B & AB & A^2B & \cdots & A^{n-1}B \end{bmatrix}$$

is of full rank.

Now the concept of observability is defined.

Definition 6. *A discrete LTI system (2.1) is called observable at $t = 0$ if the initial state x(0) can be uniquely reconstructed from a known restricted input-output trajectory. Otherwise, the system is said to be unobservable.*

Using time-invariance of (2.1) it can be shown that if the system is observable at $t = 0$ then for every $t \in \mathbb{Z}_{\geq 0}$ the state $x(t)$ is reconstructible. This result is important to control theory when it comes to state estimation (for example Kalman filter) because it gives a necessary condition for exact state estimation.
Now the test for determining observability will be stated.

Theorem 2. *(Kalman observability theorem) The pair (C, A) is observable if and only if the observability block matrix*

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{n-1} \end{bmatrix}$$

is of full rank.

The Kalman criteria for controllability/observability were stated but, there are more different equivalent statements of which the most used ones are the Popov-Belevitch-Hautus (PBH) tests.
From Theorems 1 and 2, a duality between observability and controllability matrices can be seen and it is not hard to show that using matrix operations we have:

$$(A, B) \text{ is controllable } \iff (B^T, A^T) \text{ is observable.}$$

Similarly,

$$(C, A) \text{ is observable } \iff (A^T, C^T) \text{ is controllable.}$$

2.1.3 Persistency of Excitation

For the Willems' Fundamental Lemma to hold we need a so-called persistently exciting input. By that it means a rich and diverse enough input such that all modes of the system are expressed in the output.

Hankel Matrices

For a sequence of vectors from time $t = i$ to $t = j$ we use the following vector notation:

$$s_{[i,j]} = \begin{bmatrix} s(i) \\ s(i+1) \\ \vdots \\ s(j) \end{bmatrix}$$

Where instead of vector s we will use inputs(u), states(x) and outputs(y).
Hankel matrix is a matrix whose entries are constant on the anti-diagonals, contrast to Toeplitz matrix whose entries are constant on the diagonals. Informally, Hankel matrix is a mirror image of a Toeplitz matrix.

Definition 7. *Given an $T \in \mathbb{N}$, lets consider a restricted input-output trajectory ($u_{[0,T-1]}$, $y_{[0,T-1]}$) of (2.1). Hankel matrices of depth $L \in \{1, ..., T\}$ of these inputs and outputs are given by:*

$$H_L(u_{[0,T-1]}) = \begin{bmatrix} u(0) & u(1) & \cdots & u(T-L) \\ u(1) & u(2) & \cdots & u(T-L+1) \\ \vdots & \vdots & & \vdots \\ u(L-1) & u(L) & \cdots & u(T-1) \end{bmatrix}$$

$$H_L(y_{[0,T-1]}) = \begin{bmatrix} y(0) & y(1) & \cdots & y(T-L) \\ y(1) & y(2) & \cdots & y(T-L+1) \\ \vdots & \vdots & & \vdots \\ y(L-1) & y(L) & \cdots & y(T-1) \end{bmatrix}$$

Generally, we stack input and output Hankel matrices to obtain a single block Hankel matrix used for computation:

$$\begin{bmatrix} H_L(u_{[0,T-1]}) \\ H_L(y_{[0,T-1]}) \end{bmatrix} \in M_{(m+p)L\times(T-L+1)}(\mathbb{R})$$

It is now obvious that every column of the matrices from (7) is a restricted input or output trajectory, depending on which matrix is chosen.
Now a precise definition of persistently exciting input is given.

Definition 8. *Given an sequence of inputs $u_{[0,T-1]}$ and $k \in \{1,...,T\}$. We say that the given sequence of inputs is persistently exciting of order k if the Hankel matrix $H_k(u_{[0,T-1]})$ is of full row rank.*

We have layed out all the necessary definitions and theorems needed for the statement of the main topic of this thesis, the Lemma.

2.2 The Lemma

The main result of the Lemma is that every restricted input-output trajectory of length L of the system can be obtained by linear combination of finitely many trajectories which span the whole trajectory space i.e. the behaviour.

Theorem 3. *(Willems' Fundamental Lemma) Given a LTI system (2.1), assume the pair (A,B) is controllable and let $(u_{[0,T-1]}, y_{[0,T-1]})$, $T \in \mathbb{N}$ be it's restricted input-output trajectory. Let $L \in \{1,...,T\}$, if the input $u_{[0,T-1]}$ is persistently exciting of order $N+L$ then: $(\bar{u}_{[0,L-1]}, \bar{y}_{[0,L-1]})$ is a restricted input-output trajectory of (2.1) if and only if*

$$\begin{bmatrix} \bar{u}_{[0,L-1]} \\ \bar{y}_{[0,L-1]} \end{bmatrix} = \begin{bmatrix} H_L(u_{[0,T-1]}) \\ H_L(y_{[0,T-1]}) \end{bmatrix} g$$

for some $g \in \mathbb{R}^{T-L+1}$.

Proof. Solving the difference equations of (2.1) we obtain the following result:

$$y(t) = CA^t x(0) + \left(\sum_{k=0}^{t-1} CA^{t-k-1}Bu(k)\right) + Du(t) \tag{2.2}$$

where $x(0) \in \mathbb{R}^n$ is the initial state. Generally, this could be adapted to any initial state $x_{ini} = x(t_{ini})$ using the time-invariance of the system. Now for the input-output trajectory $(\bar{u}_{[0,L-1]}, \bar{y}_{[0,L-1]})$ we can write the result from above in matrix notation like this:

$$\begin{bmatrix} \bar{y}(0) \\ \bar{y}(1) \\ \bar{y}(2) \\ \vdots \\ \bar{y}(L-1) \end{bmatrix} = \underbrace{\begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{L-1} \end{bmatrix}}_{\Omega_L} x_{ini} + \underbrace{\begin{bmatrix} D & & & & \\ CB & D & & & \\ CAB & CB & D & & \\ \vdots & \ddots & \ddots & \ddots & \\ CA^{L-2}B & \dots & \dots & \dots & D \end{bmatrix}}_{\Theta_L} \begin{bmatrix} \bar{u}(0) \\ \bar{u}(1) \\ \bar{u}(2) \\ \vdots \\ \bar{u}(L-1) \end{bmatrix}$$

Matrix Ω_L is the classical observability matrix and Θ_L is the block Toeplitz matrix of, so called, Markov parameters. Now we can reformulate the last expression like this:

$$\begin{bmatrix} \bar{u}_{[0,L-1]} \\ \bar{y}_{[0,L-1]} \end{bmatrix} = \begin{bmatrix} 0 & I \\ \Omega_L & \Theta_L \end{bmatrix} \begin{bmatrix} x_{ini} \\ \bar{u}_{[0,L-1]} \end{bmatrix} \tag{2.3}$$

Since on the left side is a general input-output trajectory of length L (assuming time-invariance), we conclude that the space of all such trajectories is spanned by the columns of the $M := \begin{bmatrix} 0 & I \\ \Omega_L & \Theta_L \end{bmatrix}$ i.e. that space is the image of that matrix. By setting the RHS of (2.3) equal to 0 and applying the rank-nullity theorem (R-N) we get:

$$\bar{u}_{[0,L-1]} = 0, \ \Omega_L x_{ini} = 0;$$

$$\begin{aligned} \implies \dim(\ker M) &= \dim(\ker \Omega_L) \underbrace{=}_{\text{R-N}} n - \text{rank } \Omega_L \\ \underbrace{\implies}_{\text{R-N}} \text{rank } M &= (n + mL) - \dim(\ker M) = \\ &= (n + mL) - (n - \text{rank } \Omega_L) = \\ &= \text{rank } \Omega_L + mL \end{aligned}$$

The columns of the Hankel matrix $\begin{bmatrix} H_L(u_{[0,T-1]}) \\ H_L(y_{[0,T-1]}) \end{bmatrix}$ are input-output trajectories so we conclude that:

$$im \begin{bmatrix} H_L(u_{[0,T-1]}) \\ H_L(y_{[0,T-1]}) \end{bmatrix} \subseteq im \begin{bmatrix} 0 & I \\ \Omega_L & \Theta_L \end{bmatrix}$$

To prove the Lemma we need to prove the equality of those spaces. We assume the strict inclusion:

$$im \begin{bmatrix} H_L(u_{[0,T-1]}) \\ H_L(y_{[0,T-1]}) \end{bmatrix} \subsetneq im \begin{bmatrix} 0 & I \\ \Omega_L & \Theta_L \end{bmatrix}$$

The expression (2.3) can be used to formulate an analog expression using Hankel matrices like this:

$$\begin{bmatrix} H_L(u_{[0,T-1]}) \\ H_L(y_{[0,T-1]}) \end{bmatrix} = \begin{bmatrix} 0 & I \\ \Omega_L & \Theta_L \end{bmatrix} \begin{bmatrix} X_{[0,T-L]} \\ H_L(u_{[0,T-1]}) \end{bmatrix}$$

Where $X_{[0,T-L]} := [x(0) \ x(1) \ ... \ x(T-L)]$. By our assumption the matrix $J_L := \begin{bmatrix} X_{[0,T-L]} \\ H_L(u_{[0,T-1]}) \end{bmatrix}$ must not have full row rank, if it were full row rank than the rank of Hankel matrix on the left would be equal to the rank of J_L and thus the image spaces would be equal. That implies that the left kernel of J_L is non-trivial, in other words, there exists a non-zero vector $v = (a^T, b_0^T, b_1^T, ..., b_{L-1}^T)$, $a \in \mathbb{R}^n$, $b_i \in \mathbb{R}^{mL}$ such that $vJ_L = 0$. Expanding we get:

$$a^T x(t) + \sum_{k=0}^{L-1} b_k^T u(t+k) = 0, \quad \forall t \in \{0, 1, ..., T-L\} \tag{2.4}$$

Shifting the above equation by an arbitray $j \in \mathbb{N}$ and solving the difference equations of (2.1) for the shifted state (similar to (2.3)) we obtain:

$$a^T A^j x(t) + \sum_{i=0}^{j-1} a^T A^{j-i-1} B u(t+i) + \sum_{k=0}^{L-1} b_k^T u(t+k+j) = 0 \tag{2.5}$$

We want to remove the state $x(t)$ from the equation, the easiest way to do it is to leverge the Cayley-Hamilton theorem which states that matrix A satisfies it's characteristic polynomial $p(\lambda) = \lambda^n + \alpha_{n-1}\lambda^{n-1} + ... + \alpha_1\lambda + \alpha_0$;

$$p(A) = A^n + \alpha_{n-1}A^{n-1} + ... + \alpha_1 A + \alpha_0 I = 0$$

Now, if we take $n+1$ copies of equation (2.5) for each $j = 0, 1, ..., n$, and multiply each one with its corresponding α_j and and sum them all, the terms with $x(t)$ cancel as a result of C-H theorem. Now we get the following expression with only the inputs:

$$\sum_{l=0}^{n+L-1} \beta_l u(t+l) = 0, \quad \forall t \in \{0, 1, ..., T-L\} \tag{2.6}$$

The expression for β_l-s can be computed by equating terms with the sum in which C-H was used and the result is:

$$\beta_l := \sum_{j=max(0,l-L+1)}^{min(n,l)} \alpha_j b_{l-j}^T + a^T \sum_{j=l+1}^{n} \alpha_j A^{j-l-1} B \tag{2.7}$$

Equation (2.6), by the definition of a Hankel matrix, can be rewritten as:

$$\begin{bmatrix} \beta_0 & \beta_1 & \dots & \beta_{n+L-1} \end{bmatrix} H_{n+L}(u_{[0,T-1]}) = 0$$

Since, $u_{[0,T-1]}$ is persistently exciting of order $N+L$, and $N \geq n$, it is clear that $H_{n+L}(u_{[0,T-1]})$ is of full row rank so we conclude that:

$$\beta_0 = \beta_1 = ... = \beta_{n+L-1} = 0$$

From (2.7) if $l \geq n$ the right sum is empty so $\beta_n, ..., \beta_{n+L-1}$ depend only on b_k-s. Now we get:

$$\beta_{n+L-1} = \alpha_n b_{L-1}^T \underbrace{=}_{\alpha_n = 1} b_{L-1}^T = 0$$

$$\beta_{n+L-2} = b_{L-2}^T + \alpha_{n-1} b_{L-1}^T = b_{L-2}^T = 0$$

continuing until $l = n$ we get $b_0 = b_1 = ... = b_{L-1} = 0$ so the left sum of (2.7) is 0 and thus for $l \leq n$ the right sum remains. Now:

$$\beta_{n-1} = \alpha_n a^T B = a^T B = 0$$

$$\beta_{n-2} = a^T(AB + \alpha_{n-1}B) = a^T AB + a^T \alpha_{n-1} B \implies a^T AB = 0$$

continuing until $l = 0$ we obtain the following result:

$$a^T \underbrace{\begin{bmatrix} B & AB & \dots & A^{n-1}B \end{bmatrix}}_{\mathcal{C}} = 0$$

Since (A, B) is controllable then $\mathcal{C}$ is of full rank, and thus $a = 0 \implies v = 0$. Which contradicts our assumption. Thus:

$$im \begin{bmatrix} H_L(u_{[0,T-1]}) \\ H_L(y_{[0,T-1]}) \end{bmatrix} = im \begin{bmatrix} 0 & I \\ \Omega_L & \Theta_L \end{bmatrix}$$

Which means there exists a $g \in \mathbb{R}^{T-L+1}$ such that for any restricted input-output trajectory $(\bar{u}_{[0,L-1]}, \bar{y}_{[0,L-1]})$ we have:

$$\begin{bmatrix} \bar{u}_{[0,L-1]} \\ \bar{y}_{[0,L-1]} \end{bmatrix} = \begin{bmatrix} H_L(u_{[0,T-1]}) \\ H_L(y_{[0,T-1]}) \end{bmatrix} g$$

□

Technically, the proof above is just the ($\Longrightarrow$) direction, however, since each column of the stacked Hankel matrix is an restricted input-output trajectory the ($\Longleftarrow$) direction follows directly from that. Stability of the system is not needed because the Lemma uses finite-horizon trajectories and thus doesn't require boundedness of state as $t \to \infty$, also the observability is not needed as it can be seen in the formulation and proof. In the proof we have also showed that the matrix:

$$\begin{bmatrix} X_{[0,T-L]} \\ H_L(u_{[0,T-1]}) \end{bmatrix}$$

is of full rank. Also we have made an indirect assumption regarding row rank for PE. For a matrix to have full row rank, a necessary condition is that the number of columns needs to be atleast equal to the number of rows, thus PE condition needs that condition fullfilled. When we impose that condition on $H_{N+L}(u_{[0,T-1]})$ we get that:

$$T - (N + L + 1) \geq m(N + L) \iff T \geq (m + 1)(N + L) - 1 \tag{2.8}$$

So for PE the length of initial trajectory T needs to satisfy the condition above for a given L. Generally, for a long enough recorded trajectory we can use the Lemma for a shorter length trajectory. In our Lemma formulation we only used trajectories of length L starting from 0, but it can be shown that the Lemma holds for a general restricted input output trajectory of length L, i.e. for time window of $[i, i + L - 1]$ $\forall i \in \mathbb{Z}_{\geq 0}$ we have:

$$\begin{bmatrix} \bar{u}_{[i,i+L-1]} \\ \bar{y}_{[i,i+L-1]} \end{bmatrix} = \begin{bmatrix} H_L(u_{[0,T-1]}) \\ H_L(y_{[0,T-1]}) \end{bmatrix} g \tag{2.9}$$

Now we will define the lag ℓ of the system. Smallest positive integer k s.t. rank Ω_k = rank Ω_{k+1}, where Ω_k is the observability matrix from (2.3), is called the lag of the system, in other words, the smallest k after which the rank of the observability matrix does not change. Clearly, by C-H theorem, $\ell \leq n$. For an observable system that is the minimal trajectory length such that the initial state x_{ini} can be reconstructed.

3 Lemma - Numerical Examples

In this chapter we will demonstrate the Fundamental Lemma on concrete linear systems. We will use the system model to gather the necessary data for the Hankel matrices and then use them to verify for a random trajectory that it lies in the image of the Hankel matrix. For this we use MATLAB because it is easier to work with its Control System Toolbox for system simulation. TODO:text

3.1 Simple SISO System

Consider a discrete time LTI system (2.1) with state space matrices:

$$A = \begin{bmatrix} 0.5 & 0.1 & 0.1 \\ 0.1 & 0.4 & 0.2 \\ -0.1 & 0.2 & 0.3 \end{bmatrix} \quad B = \begin{bmatrix} -1 \\ 0.1 \\ 0.7 \end{bmatrix} \tag{3.1}$$

$$C = \begin{bmatrix} 0.1 & 2 & 1.5 \end{bmatrix} \qquad D = \begin{bmatrix} 0 \end{bmatrix} \tag{3.2}$$

Clearly $n = 3, m = p = 1$ and the eigenvalues of A are $\{0.5834,\ 0.4657,\ 0.1509\}$ which all have their absolute value less then 1 which means the system is stable. Also the controllability and observability matrix have a non-zero determinant to the system is both controllable and observable, which means, it is suitable for the Fundamental Lemma. For this example the length of initial trajectory and length of trajectories for reconstruction are chosen as $T = 50, L = 22$ it can be checked that for $N = n$ the (2.8) is satisfied. For the input for this example we are using a randomly generated input using MATLAB's *rand* function. Since the input is random there is a high chance for persistency of excitation before testing the lemma we check the rank of the $H_{N+L}(u_{[0,T-1]})$. We input the noisy input to the system and record the output data, we then create the Hankel matrix $\begin{bmatrix} H_L(u_{[0,T-1]}) \\ H_L(y_{[0,T-1]}) \end{bmatrix}$. Now we generate a new random input-output trajectory of the system $(\bar{u}_{[0,L-1]}, \bar{y}_{[0,L-1]})$. We now use the Hankel matrix and the new data to find the vector g. We impose that as a least-squares problem since the Hankel matrix is generally not square and the system of equations is underdetermined (fewer rows than columns). The objective for a minimization problem is the following:

$$\text{Find } g \in \mathbb{R}^{T-L-1} \text{ that minimizes } \left\| \begin{bmatrix} H_L(u_{[0,T-1]}) \\ H_L(y_{[0,T-1]}) \end{bmatrix} g - \begin{bmatrix} \bar{u}_{[0,L-1]} \\ \bar{y}_{[0,L-1]} \end{bmatrix} \right\|_2^2 \tag{3.3}$$

In MATLAB we solve the following problem using the *lsqminnorm* function, which uses *complete orthogonal decomposition (COD)* rather than *singular value decomposition (SVD)* to compute g, however, for our example both SVD and COD should yield similar results. Now the Lemma states that g should make that norm of the difference equal to zero. That means the g is an exact solution, not an approximation. Computing the norm for the calculated g we get result in MATLAB:

$$\left\| \begin{bmatrix} H_L(u_{[0,T-1]}) \\ H_L(y_{[0,T-1]}) \end{bmatrix} g - \begin{bmatrix} \bar{u}_{[0,L-1]} \\ \bar{y}_{[0,L-1]} \end{bmatrix} \right\| \leq 10^{-10} \tag{3.4}$$

The norm is not exactly 0 but it is really close, the small error arises from truncation of real numbers due to floating point arithmetic, so we can assume the result is valid. Our reconstructed trajectory is found in the image of the Hankel matrix created from recorded data.

3.2 Mechanical MIMO System

In this example we will demonstrate the Lemma on a concrete mechanical system. We will show, like in the last example, that we can reconstruct a trajectory from previously recorded data.

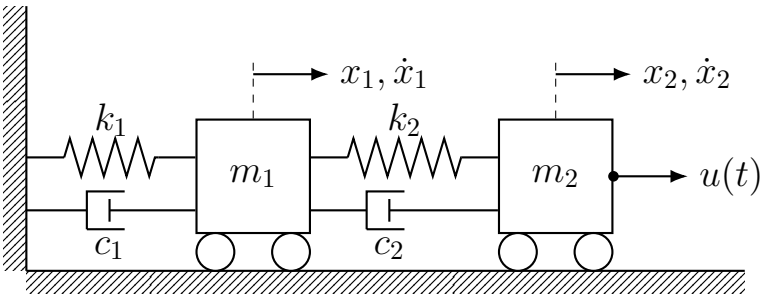


Figure 3.1: Double spring–mass–damper system with force input acting on second mass.

The states of the system are positions and velocities of each mass, the input is a force on the second mass. We chose the output to be a vector of positions of both mass thus this is a MIMO system. Using mechanical equilibrium equations we can get a system of coupled linear differential equations which can be represented as a continuous state space LTI system like this:

$$\dot{x}(t) = Ax(t) + Bu(t)$$
$$y(t) = Cx(t) + Du(t)$$

Where (for this concrete example: $m_1 = 1.0kg$, $m_2 = 0.8kg$, $k_1 = 120Nm^{-1}$, $k_2 = 180Nm^{-1}$, $c_1 = 1.5Nsm^{-1}$, $c_2 = 2.0Nsm^{-1}$):

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ -\frac{k_1+k_2}{m_1} & -\frac{c_1+c_2}{m_1} & -\frac{k_2}{m_1} & -\frac{c_2}{m_1} \\ 0 & 0 & 0 & 1 \\ \frac{k_2}{m_2} & \frac{c_2}{m_2} & -\frac{k_2}{m_2} & -\frac{c_2}{m_2} \end{bmatrix} \quad B = \begin{bmatrix} 0 \\ 0 \\ 0 \\ \frac{1}{m_2} \end{bmatrix} \tag{3.5}$$

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \qquad D = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \tag{3.6}$$

This is a continuos state-space model but we need a discrete model so we need to discretize this. There are many ways of discretizing the continuous model, we will derive the discrete version matrices A_d, B_d, C_d, D_d using zero order hold (ZOH). Assume sampling period T_s, we chose $T_s = 10^{-2}s$. Now we will begin from the solution of the general continuous state space system:

$$x(t) = e^{A(t-t_0)}x(t_0) + \int_{t_0}^{t} e^{A(t-\tau)}Bu(\tau)d\tau$$

Let $k \in \mathbb{Z}_{\geq 0}$ and $t_0 = kT_s,\ t = (k+1)T_s$. Define $x_k := x(kT_s),\ u_k := u(kT_s)$. Now we have:

$$x_{k+1} = e^{AT_s}x_k + \int_{kT_s}^{(k+1)T_s} e^{A[(k+1)T_s - \tau]}Bu(\tau)d\tau$$

Now assume $u(\tau) = u(kT_s) = u_k = const.\ \forall\tau \in (kT_s, (k+1)T_s),\ \forall k \in \mathbb{Z}_{\geq 0}$. This is the ZOH, we assume the input is constant between sampling. After a change of variables for integration we arrive at this result since B and u_k are constants they can go outside the integral.

$$x_{k+1} = e^{AT_s}x_k + \left(\int_0^{T_s} e^{Az}dz\right)Bu_k$$
$$y_k = Cx_k + Du_k$$

We can now see that this is essentially (2.1) but where:

$$A_d = e^{AT_s},\ B_d = \left(\int_0^{T_s} e^{Az}dz\right)B,\ C_d = C,\ D_d = D$$

This was a general derivation, in MATLAB we use the function *c2d* to convert the system from continuos to discrete time. Now we can see that $n = 4,\ m = 1,\ p = 2$. The initial trajectory we recorded is 30 seconds long, in MATLAB this was converted to number of samples with $T = round(30/Ts)$. The length of reconstructed trajectories L is 12 seconds. We set $N = n$ because the systems has four energy tanks. For the input, instead of a random sequence of real numbers between 0 and 1, we use a pseudo-random binary sequence (PRBS) - a binary signal which jumps randomly between +1 and -1, we multiply that signal with a positive real number (amplitude) and feed it through a low-pass filter to make it a bit smoother and more realistic. This choice of T, L and N satisfy the (2.8).

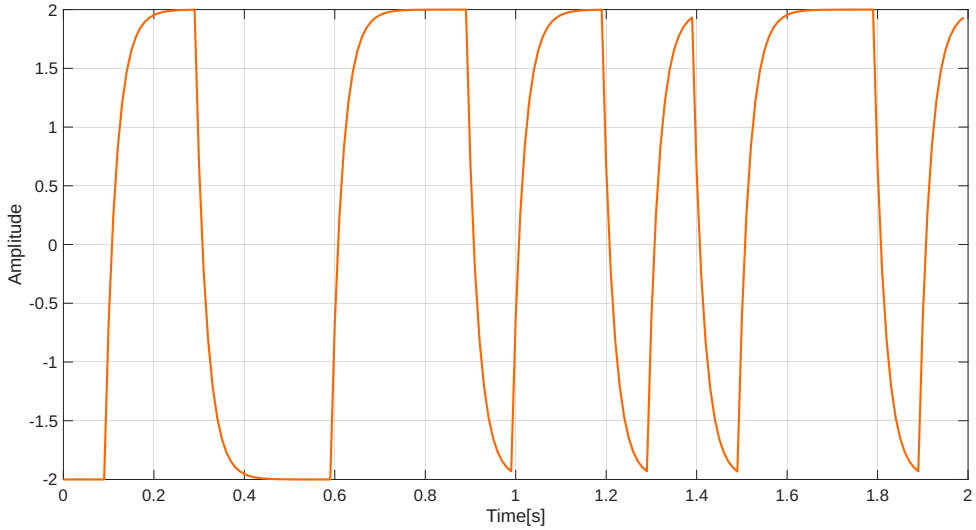
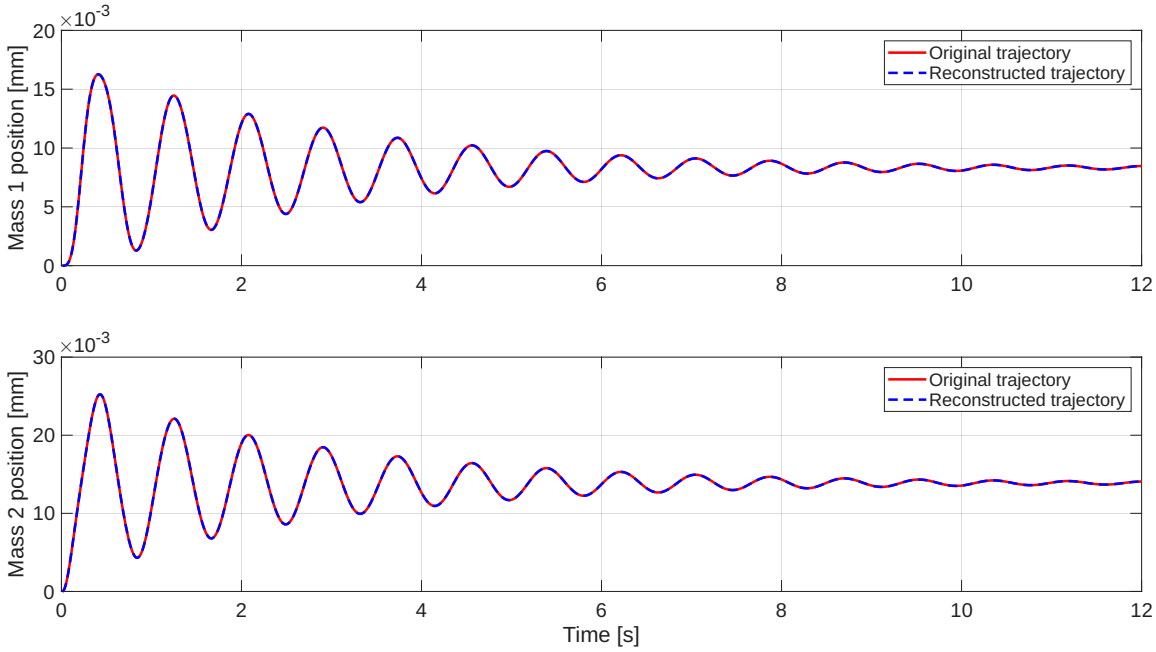


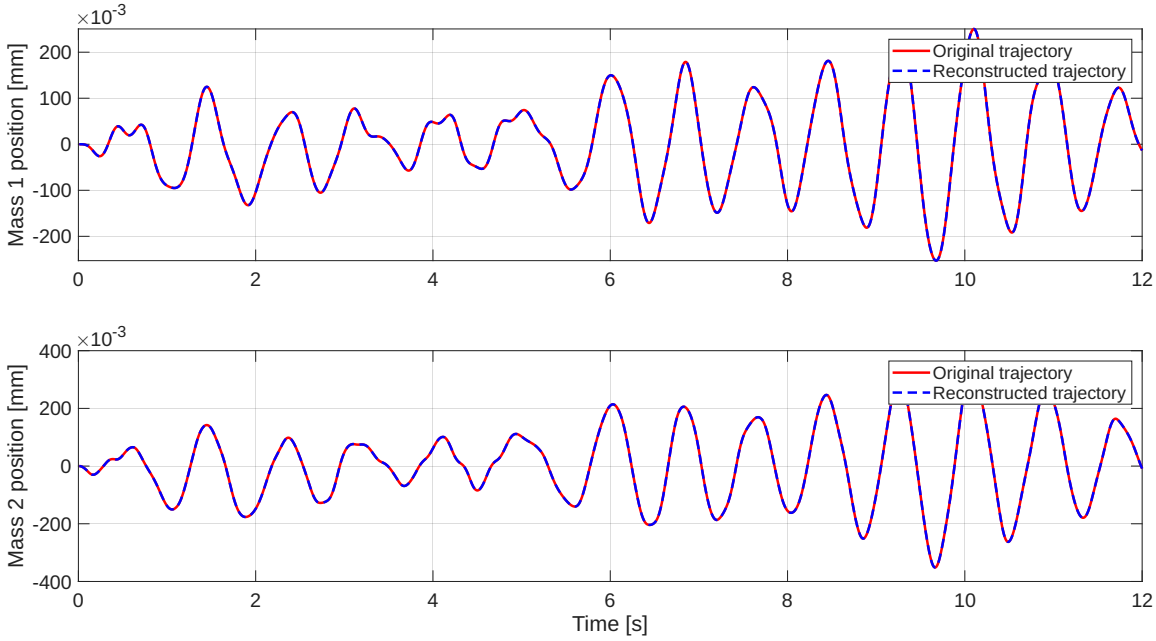
Figure 3.2: Low-pass filtered PRBS input signal

Since this type of input data has enough "randomnes" it is a good choice for a persistently exciting input. And after testing the rank condition for PE it was always satisfied.

Now, as before, we input that PE signal to the system and place the recorded trajectories into the according Hankel matrix. We will test the Lemma on two inputs, first will be a step function and second will be a new PRBS signal. We simulate the output trajectories using the system model and solve for vector g like in (3.3).



(a) Output trajectories for a unit step input



(b) Output trajectories for a PRBS input

Figure 3.3: Comparison of original and reconstructed trajectories of the system

It can be seen that trajectories line-up, this is confirmed numerically with (3.4) seeing that for both trajectories we have the norm $\leq 10^{-10}$. Thus, the Lemma holds for this example.

4 Data driven simulation of LTI systems

In the last chapter we used the FL to demonstrate that any length L trajectory of a linear system can be found in the image of a Hankel matrix that is built using a previous trajectory with a PE input. This is really interesting, however, there are no direct use cases when it comes to control. In this chapter we will build up on the FL and use it to simulate a future of a given trajectory using the same Hankel matrix.

4.1 Problem statement and solution

Take the system (2.1) and assume (A, B) is controllable. Let $L_{ini} \in \mathbb{Z}_{\geq 0}$ be the past length of the trajectory we want to simulate and $L_{ref} \in \mathbb{Z}_{\geq 0}$ be the length of the future part. Let $L := L_{ini} + L_{ref}$. Assume the following are given:

- Upper bound on state dimension $N \geq n$
- Restricted input-output trajectory $(u_{[0,T-1]}, y_{[0,T-1]})$, where T satisfies (2.8)
- An initial restricted input-output trajectory $(\bar{u}_{[0,L_{ini}-1]}, \bar{y}_{[0,L_{ini}-1]})$
- Reference input $\bar{u}_{[L_{ini},L-1]}$

We want to find the output $\bar{y}_{[L_{ini},L-1]}$ such that $(\bar{u}_{[0,L-1]}, \bar{y}_{[0,L-1]})$ is a restricted input-output trajectory of (2.1). So based on recorded data, previous history of a trajectory and future input we want to find the future output part of that trajectory. We can split the Hankel matrices of recorded data like this:

$$\begin{bmatrix} H_L(u_{[0,T-1]}) \\ H_L(y_{[0,T-1]}) \end{bmatrix} = \begin{bmatrix} U_p \\ U_f \\ Y_p \\ Y_f \end{bmatrix} \tag{4.1}$$

Where U_p and U_f have mL_{ini} and mL_{ref} and Y_p and Y_f have pL_{ini} and pL_{ref} rows, respectively. Now we will formulate a solution of the problem like in [TODO: ref DBCBook].

Theorem 4. *Consider a system (2.1) with (A, B) controllable and PE input $u_{[0,T-1]}$ of order $N+L$, assume restricted input output trajectory $(\bar{u}_{[0,L_{ini}-1]}, \bar{y}_{[0,L_{ini}-1]})$ and reference input $\bar{u}_{[L_{ini},L-1]}$. Then the following statements hold:*

(i) There exists $g \in \mathbb{R}^{T-L+1}$ such that:

$$\begin{bmatrix} U_p \\ Y_p \\ U_f \end{bmatrix} g = \begin{bmatrix} \bar{u}_{[0,L_{ini}-1]} \\ \bar{y}_{[0,L_{ini}-1]} \\ \bar{u}_{[L_{ini},L-1]} \end{bmatrix}$$

(ii) If g is a solution to (i) then define the output $\bar{y}_{[L_{ini},L-1]} = Y_f g$. Such defined output makes $(\bar{u}_{[0,L-1]}, \bar{y}_{[0,L-1]})$ a restricted input-output trajectory of (2.1)

(iii) If $L_{ini} \geq \ell$ than such output in (ii) is the only one that makes $(\bar{u}_{[0,L-1]}, \bar{y}_{[0,L-1]})$ a restricted input-output trajectory, i.e. it is unique.

4.2 Simulating LTI system from data

This theorem gives us the method on how to simulate the future trajectory of the system. We will test this result on examples. Firstly, we will test try to simulate future output trajectory for the LTI MIMO example from the previous chapter (3.1). We simulated the system with a PRBS input for 50 seconds ($T_s = 0.01s$ $T = 5000$) and recorded the trajectory $(u_{[0,T-1]}, y_{[0,T-1]})$. We chose the following lengths of past and future part of to be simulated trajectory:

$$L_{ini} = 500,\ L_{ref} = 1000,\ L = L_{ini} + L_{ref} = 1500$$

We populated the $H_L(u_{[0,T-1]})$ and $H_L(y_{[0,T-1]})$ and split them into (U_p, Y_p, U_f, Y_f) parts. Result from (4, (i)) tells us g exists, we determine it using the modified least-squares problem from the last chapter (3.3):

$$\text{Find } g \in \mathbb{R}^{T-L-1} \text{ that minimizes } \left\| \begin{bmatrix} U_p \\ Y_p \\ U_f \end{bmatrix} g - \begin{bmatrix} \bar{u}_{[0,L_{ini}-1]} \\ \bar{y}_{[0,L_{ini}-1]} \\ \bar{u}_{[L_{ini},L-1]} \end{bmatrix} \right\|_2^2 \tag{4.2}$$

Solving with MATLAB's *lsqminnorm* we obtain the vector g. Using g we calculate the future output trajectory $\bar{y}_{[L_{ini},L-1]} = Y_f g$. By theorem (4) this output part is unique since $L_{ini} \geq \ell = 2$ (for the parameters given in (3.5)), generally, the lag is unknown, but if we take large enough L_{ini} we can be sure that it is larger. Now we will compare the data based future output trajectories with one calculated using the system model - the first L_{ini} points using the model. For the input signal we used a multi-sine input like this:

$$u(k) = 10\sin(2\pi k T_s) + \frac{1}{3}\sin(2\pi 3 k T_s),\ k \in \{0, 1, \ldots, L-1\}$$

The simulation output is:

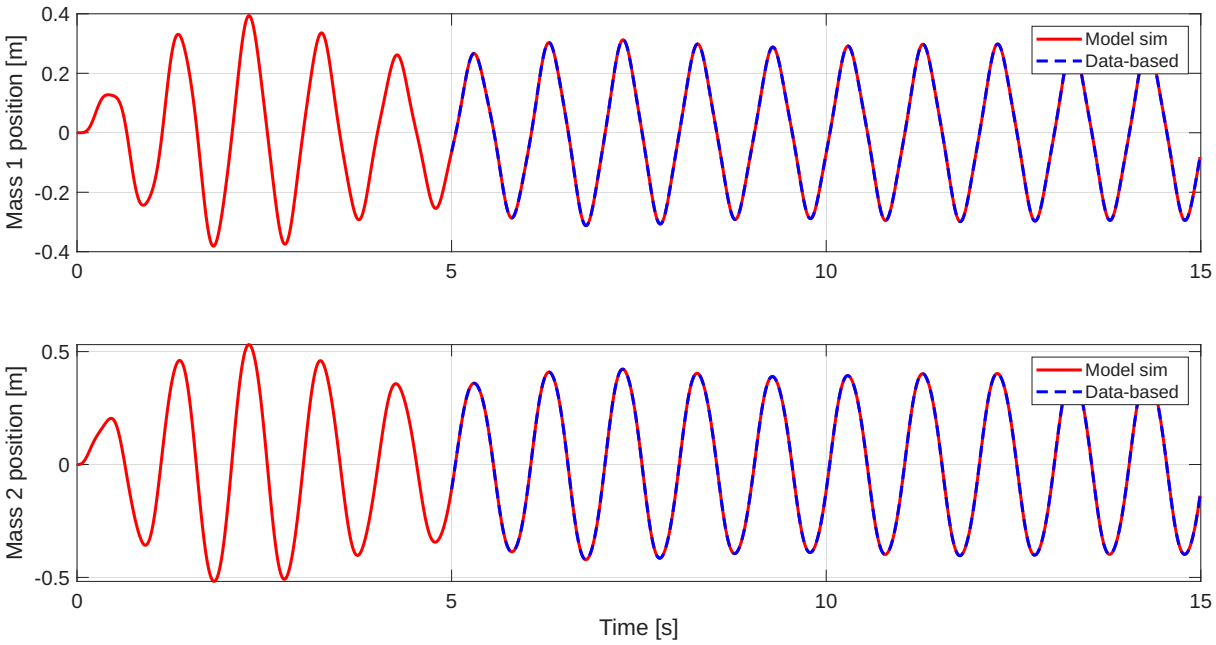


Figure 4.1: Data-based vs. model-based future trajectory simulation

We can see that the data-based simulated trajectories perfectly match the model-based ones. Numerically, like before, we get the following confirmation that the future output trajectories coincide:

$$\left\| \bar{y}_{[L_{ini},L-1]} - \bar{y}^{model}_{[L_{ini},L-1]} \right\| \leq 10^{-10}$$

4.3 Simulating a nonlinear system around equilibrium

In the last section we showed that it is possible to simulate a LTI system directly from recorded data without the model. For nonlinear systems, a classical approach is to linearize the dynamics around a stationary point around which the linearized model is mostly valid. We want to gather the data around the stationary (equilibrium) point of the system and apply the data-based simulation of the future trajectory. For this example we will take a classical torque driven pendulum:

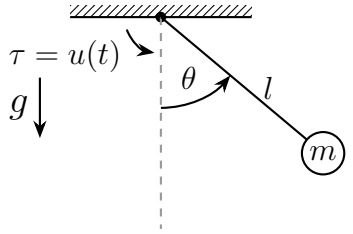


Figure 4.2: Torque driven simple pendulum

Parameters for our pendulum are:

$$m = 0.2\ kg,\ l = 0.5\ m,\ b = 0.05\ Nms,\ g = 9.81\ \frac{m}{s^2}$$

Assuming damping is proportional to angular velocity with proportionality constant b, we write the state space dynamics which can be derived with various approaches from classical mechanics, state vector is $x = (\theta,\ \dot{\theta})^T$:

$$\dot{x} = \frac{d}{dt}\begin{bmatrix}\theta \\ \dot{\theta}\end{bmatrix} = \begin{bmatrix}\dot{\theta} \\ \frac{-b}{ml^2}\dot{\theta} - \frac{g}{l}\sin(\theta) + \frac{u}{ml^2}\end{bmatrix} = f(x,u) \tag{4.3}$$

Since this is in continous state space forumlation we need to derive a discrete time version of it. Using sampling period $T_s = 0.01s$ and assuming ZOH on the input $u(t)$ (see 3.2) we discretize (4.3) using the Runge-Kutta 4 method like this:

$$x_{k+1} = x_k + \frac{T_s}{6}(k_1 + 2k_2 + 2k_3 + k_4) \tag{4.4}$$

Where,

$$\begin{aligned} k_1 &= f(x_k, u_k) \\ k_2 &= f(x_k + \frac{T_s}{2}k_1, u_k) \\ k_3 &= f(x_k + \frac{T_s}{2}k_2, u_k) \\ k_4 &= f(x_k + T_s k_3, u_k) \end{aligned}$$

Assuming initial state $x_0 = (0,0)^T$ and that the output is the full state of the system ($y_k = x_k$) we supply a PRBS input for 50 seconds to the system and record the output. We configured the amplitude of the PRBS such that angle θ is always $\leq \frac{\pi}{18}$ (10 degrees).

Since θ is small enough and the model is approximately linear around 0 we will try to simulte the future trajectory of the system using the FL. We chose the following trajectory lengths $L_{ini} = 50$, $L_{ref} = 600 \implies L = 650$, we assumed $N = 4$. Since a PRBS input is PE of order $N + L$ we can repeat the procedure in 4.2. We give a zero input and set the initial state to $\left(\frac{\pi}{10}, 0\right)^T$ for the simulating trajectory - autonomous system.

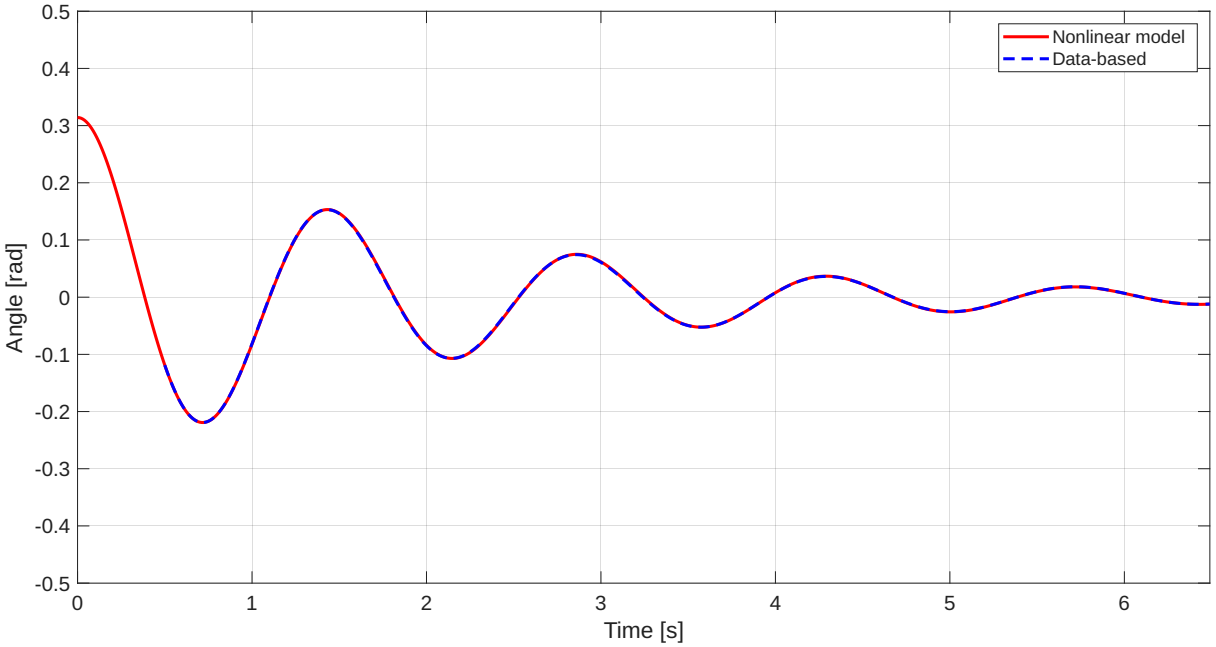


Figure 4.3: Pendulum trajectory simulation - data-based vs. model

If we now calculate the norm of the difference between the two output trajectories we get:

$$\left\| \bar{y}^{\,data}_{[L_{ini},L-1]} - \bar{y}^{\,model}_{[L_{ini},L-1]} \right\| \sim 10^{-2}$$

Our trajectories do not coincide exactly, but that is to be expected because the model is nonlinear, since the error is small we can say that data-based simulation works well for small angles. Now we will try simulating with a larger angle and we changed $L_{ini} = 10$. We will also compare a linearized model of (4.3) which uses $\sin(\theta) \approx \theta$ linearization. For initial angle of $\frac{\pi}{3}$ and zero input we get the following results (for a specific simulation):

$$\left\| \bar{y}^{\,data}_{[L_{ini},L-1]} - \bar{y}^{\,model}_{[L_{ini},L-1]} \right\| \approx 0.428618 \tag{4.5}$$

$$\left\| \bar{y}^{\,lin.model}_{[L_{ini},L-1]} - \bar{y}^{\,model}_{[L_{ini},L-1]} \right\| \approx 1.446429 \tag{4.6}$$

We can see that the error for the data-based simulation is quite smaller, which can be seen on the following plot:

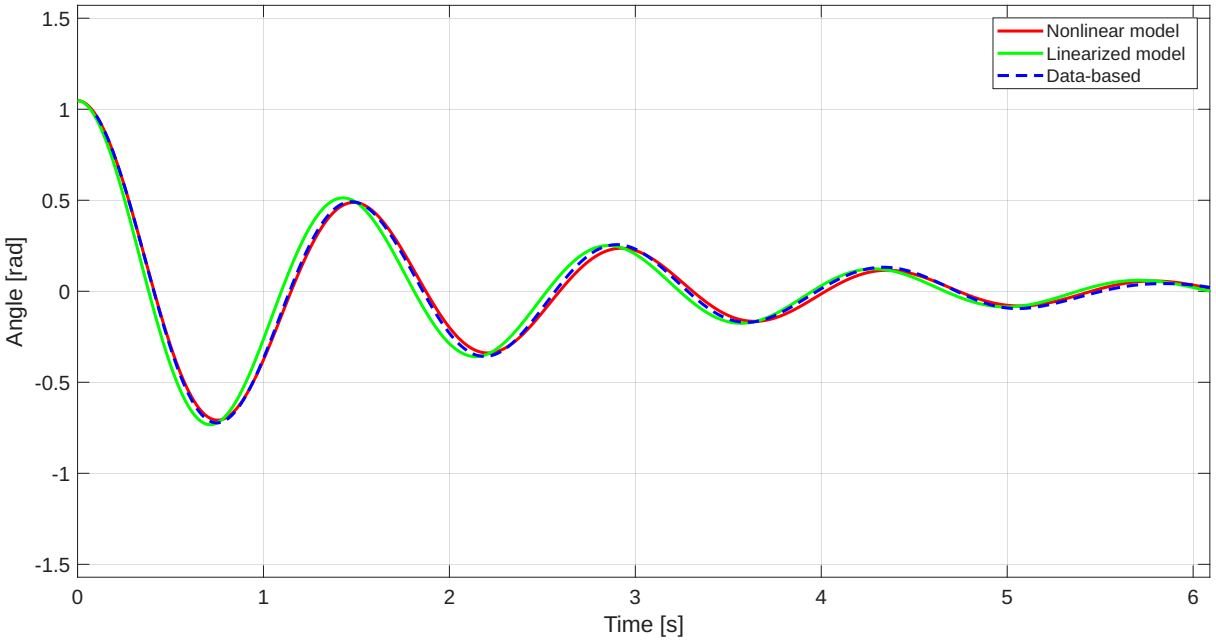


Figure 4.4: Pendulum trajectory simulation - large angles; - data-based vs. model vs. linearized model

We can see that the data-based simulation is closer to the nonlinear model one and it is considerably more accurate for larger angles. However, this MATLAB script inputs a new PRBS to the system for the data every time the script is run, randomness of that signal implies that the result from above is not general, in fact, we sometimes got that the linearized model produces a lower error than the data-based. In practice, this could be fixed by recording multiple data sets with PRBS and keeping the one which makes the data-based simulation the best. We ran the MATLAB script 1000 times and recorded the norm of the difference for each one, the linearized model error (4.6) is constant because it does not depend on a PRBS input. These are the results:

$$\left\| \bar{y}^{data}_{[L_{ini},L-1]} - \bar{y}^{model}_{[L_{ini},L-1]} \right\|_{average} = 0.685848$$
$$\left\| \bar{y}^{data}_{[L_{ini},L-1]} - \bar{y}^{model}_{[L_{ini},L-1]} \right\|_{max} = 3.47226$$
$$\left\| \bar{y}^{data}_{[L_{ini},L-1]} - \bar{y}^{model}_{[L_{ini},L-1]} \right\|_{min} = 0.238929$$

This proves that with data-based simulation we can get a better result than a linearized model, however, if we do not supply a "good" PE input we can get worse results.

In this chapter we demonstrated that it is possible to simulate future trajectories of LTI systems directly from recorded data and knowledge about the input. This hints at the following chapter which uses this approach to implement predictive control of such systems entirely from input-output data.

5 Data-Enabled Predictive Control (DeePC)

5.1 Introduction

Classical control theory almost always uses a model of the system to synthesise a controller. Usually that model is obtain by system identification from recorded data, but when the model is too complex to be easily identified or approximated from first principles people started developing control methods based solely on data. During the last decade data availability and processing power have been growing sharply thus data-driven control methods have started do develop intensively, there has been a lot of progress in the machine learning approach. In this chapter we will build up on results from previous chapters to create a controller that uses only input/output data, bypassing a model. A needed prerequisite for our data-based controller is model predictive control (MPC), an optimal control algorithm which finds optimal input to the system on a finite and receding horizon whose data is constrained with the model and other (physical) limitations. The DeePC algorithm, essentially, does the same thing as MPC - same optimization problem, just instead of the model it uses previously recorded data for the prediction part. This algorithm relies on the Fundamental Lemma since it's result states that it is possible to predict future trajectory of a linear system given input data and recorded data. First formulation (and naming) of DeePC was given by [TODO: insert reference dorfler deepc]. A good overview on data-based control that stems from the behavioural framework and the FL can be found at [TODO: special issue control systems mag ieee] and [annual reviews in control 100914]

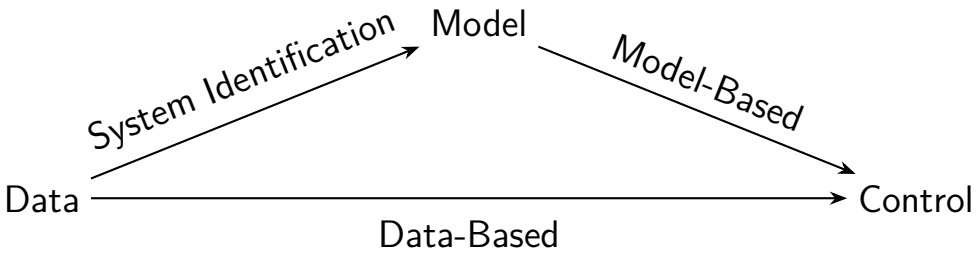
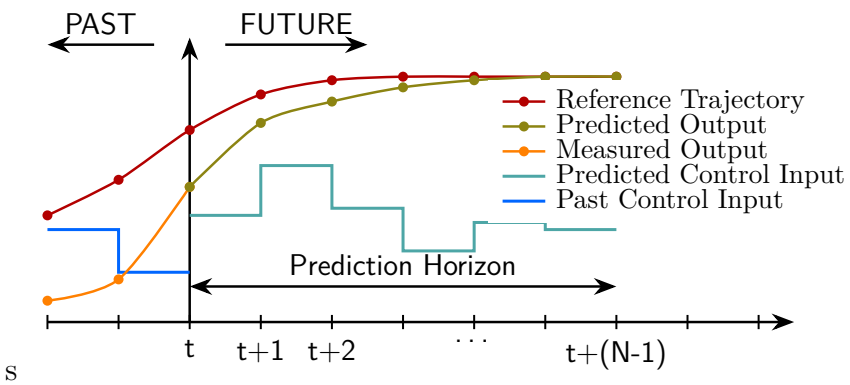


Figure 5.1: Different control design approaches from data



s

Figure 5.2: General predictive control illustration

5.2 Model predictive control

As mentioned before, MPC is a control algorithm which uses optimization with system model and limitations as constraints to find optimal control input over a finite, receding horizon. At each time step future trajectory of the system is predicted using the model and optimal sequence of control input is computed taking constraints on inputs and states of the system. Typically, only the first input of the optimal input sequence is applied after which the procedure is repeated with new measurements. We can say that, other than MPC, we do not have another control solution for multivariable systems with constraints on inputs and outputs with stability guarantees. In this section we will formulate classical MPC for LTI systems which is needed for later understanding of DeePC.

Given a LTI system (2.1) the classical finite-horizon MPC optimization problem at time step t can be written as:

$$\begin{aligned} \min_{x,y,u} \quad & \sum_{j=0}^{N-1} \left(\|y_j - r_{t+j}\|_Q^2 + \|u_j\|_R^2 \right) \\ \text{subject to:} \quad & x_0 = \hat{x}(t), \\ & x_{j+1} = Ax_j + Bu_j, \quad j = 0, \dots, N-1; \\ & y_j = Cx_j + Du_j, \quad j = 0, \dots, N-1; \\ & y_j \in \mathcal{Y}, \quad j = 0, \dots, N-1; \\ & u_j \in \mathcal{U}, \quad j = 0, \dots, N-1; \end{aligned} \tag{5.1}$$

Where: $\|z\|_P^2 := z^T P z$, for a symmetric positive semidefinite matrix P (notation $P \succeq 0$). Matrix $Q \in M_{p\times p}(\mathbb{R})$, $Q \succeq 0$ is the output cost matrix and $R \in M_{m\times m}(\mathbb{R})$, $R \succeq 0$ is the input cost matrix. Sets $\mathcal{U} \subseteq \mathbb{R}^m, \mathcal{Y} \subseteq \mathbb{R}^p$ are input and output constraints set, respectively. Sequence $(r_0, r_1, ...)$ is the reference trajectory which we want to track, $N \in \mathbb{Z}_{>0}$ is the prediction horizon length. We use estimated state $\hat{x}$ instead of the true state because, generally, the output is not the whole state i.e. we cannot measure the whole state thus we need to estimate it. This MPC optimization problem is usually a convex quadratic program (QP). Now we will lay out the MPC algorithm for reference tracking.

Algorithm 1 Model predictive control (MPC)

Require: System model (A, B, C, D); reference trajectory r; past input/output data (u, y); contraints sets $\mathcal{U}, \mathcal{Y}$; cost matrices Q, R;
for $t = 0, 1, ...$ **do**
[1] Estimate $\hat{x}(t)$ from previous input/output data
[2] Solve (5.1) for $u^* = (u_0^*, u_1^*, ..., u_{N-1}^*)$
[3] Apply input $u(t) = u_0^*$
[4] Update past input/output data with new inputs/measurements
end for

This formulation of MPC assumes stationary state inputs are 0, also terminal cost is omitted because when we transfer to DeePC we do not know the full state of the system. It can be shown, under typical assumptions, that MPC algorithm is recursively feasable and stabilizing. This algorithm can produce excellent results if the system model is precise and if it is possible to estimate the initial state $\hat{x}(t)$.

5.3 DeePC for LTI systems

In this section we will formulate predictive control based on data. We will use the same simulation method for prediction from Chapter 4. The following are required for the DeePC algorithm:

(i) An upper bound on state dimension N

(ii) Past/future trajectory lengths L_{ini} and L_{ref}, with $L_{ini} \geq \ell$; $L := L_{ini} + L_{ref}$

(iii) Recorded restricted input-output trajectory $(u_{[0,T-1]}, y_{[0,T-1]})$, where $u_{[0,T-1]}$ is PE of order N+L and $T \geq (m+1)(N+L)-1$, m is the number of inputs

(iv) Initial (past) trajectory $(\bar{u}_{ini}, \bar{y}_{ini}) = (\bar{u}_{[0,L_{ini}-1]}, \bar{y}_{[0,L_{ini}-1]})$

(v) Reference trajectory $(r_0, r_1, ...) \in (\mathbb{R}^p)^{\mathbb{Z}_{\geq 0}}$

(vi) Input and output cost matrices $Q \succeq 0,\ R \succeq 0$

(vii) Constraits sets $\mathcal{U},\ \mathcal{Y}$

Requirement (ii) assuemes initial trajectory length longer than the lag of the system which is unknown, in practice we take long enough initial trajectory. That condition is essentially equivalent to having long past input-output data for initial state estimation in MPC. Now we define matrices U_p, U_f, Y_p, Y_f like in (4.1). Conditions above allow for application of Theorem 4. We now formulate the DeePC optimization problem at time step t:

$$\begin{aligned} \min_{g,y,u} \quad & \sum_{j=0}^{L_{ref}-1} \left(\|\bar{y}_j - r_{t+j}\|_Q^2 + \|\bar{u}_j\|_R^2 \right) \\ \text{subject to:} \quad & \begin{bmatrix} U_p \\ Y_P \\ U_f \\ Y_f \end{bmatrix} g = \begin{bmatrix} \bar{u}_{ini} \\ \bar{y}_{ini} \\ \bar{u} \\ \bar{y} \end{bmatrix} \\ & \bar{y}_j \in \mathcal{Y}, \quad j = 0, \ldots, L_{ref} - 1; \\ & \bar{u}_j \in \mathcal{U}, \quad j = 0, \ldots, L_{ref} - 1; \end{aligned} \tag{5.2}$$

Where:

$$\begin{aligned} \bar{u} &= col(\bar{u}_0, \ldots, \bar{u}_{L_{ref}-1}) \\ \bar{y} &= col(\bar{y}_0, \ldots, \bar{y}_{L_{ref}-1}) \end{aligned}$$

We deviate from previos nomenclature in favor of easier formulation. Initial trajectory (u_{ini}, y_{ini}) will be updated with new meausred data such that it keeps the same length thus the oldest data pair will be removed.

We can see that the DeePC optimization problem is *de facto* a MPC problem but where instead of prediction using the model we use previous input-output data, thus this problem aviods the model completely. We now formulate the DeePC algorithm:

Algorithm 2 Data-Enabled predictive control (DeePC)

Require: Requirements (i) - (vii) listed above
for $t = 0, 1, ...$ **do**
[1] Solve (5.2) for the vector g
[2] Compute optimal input sequence $\bar{u}^* = U_f g$
[3] Apply the first input $u(t) = \bar{u}^*(0)$
[4] Update the initial trajectory (u_{ini}, y_{ini}) with new inputs/measurements
end for

It can be shown that this DeePC is equivalent to MPC under the assumptions of the Fundamental Lemma, i.e. given the same reference and initial trajectory they will give the identical output.

Now, will we try to control the double spring mass damper system 3.1 with DeePC. We will try to control the position of mass 2 so we changed the output equation ($C = \begin{bmatrix} 0 & 0 & 1 & 0 \end{bmatrix}$, $D = 0$) to only give position of mass 2 to reduce computational cost. We put an input constraint $-100 \leq u(t) \leq 100$. We chose the following parameters $T = 100$, $L_{ini} = 5$, $L_{ref} = 25$, $Q = 10000$, $R = 0.001$ - changing them we can tune the behaviour of our controller. Initial trajectory is set to 0.

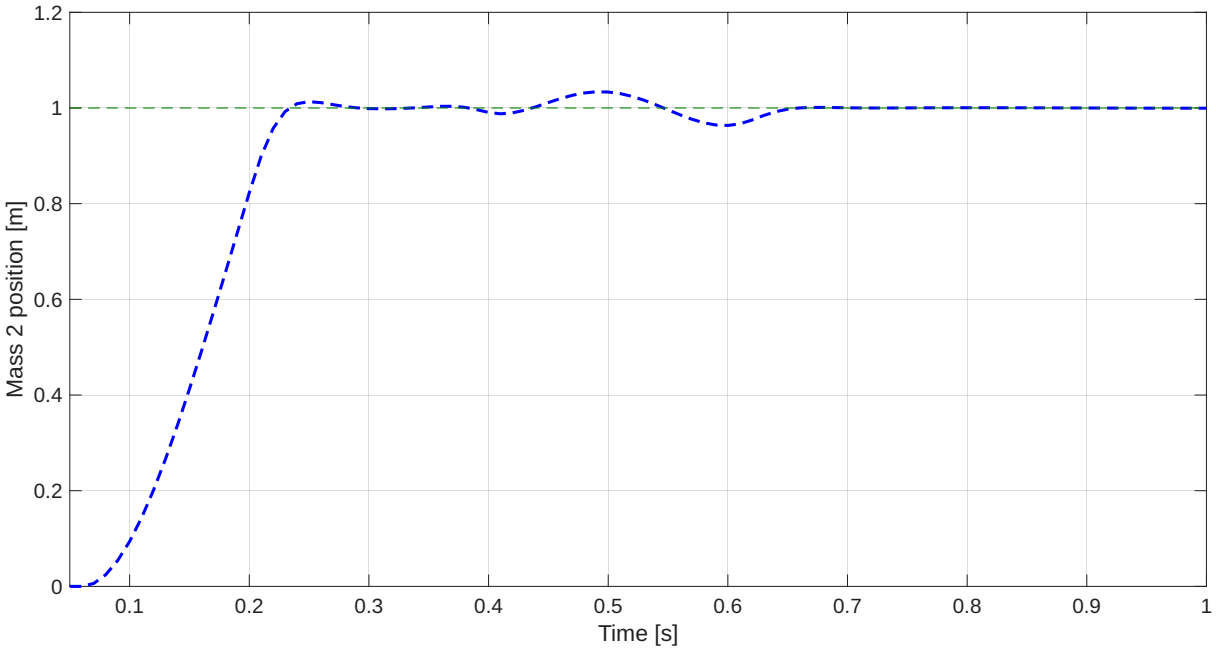


Figure 5.3: Mass 2 position controlled with DeePC

We can see that we were able to reach tracking of the given trajectory, however, it could be tuned for a better result, but as a proof of concept it is sufficient. Now lets display the limited force input.

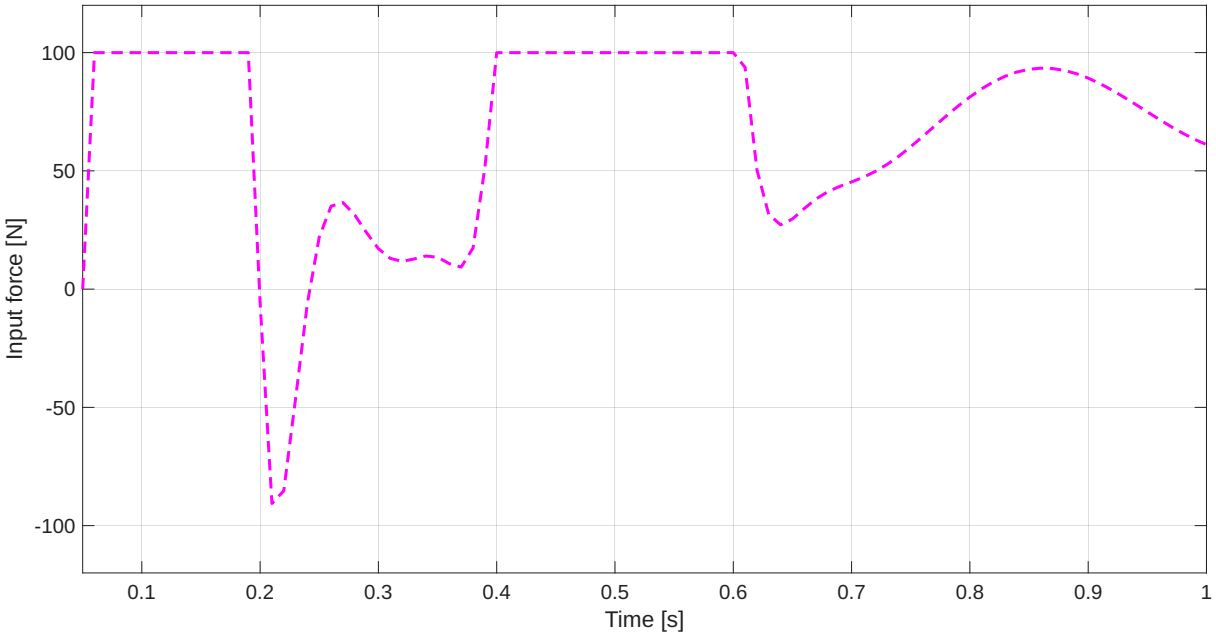


Figure 5.4: Force input for mass 2 position control with DeePC

We can see that the limit on the input is working. Now lets try limiting the position $y \leq 1$ to eliminate overshoot.

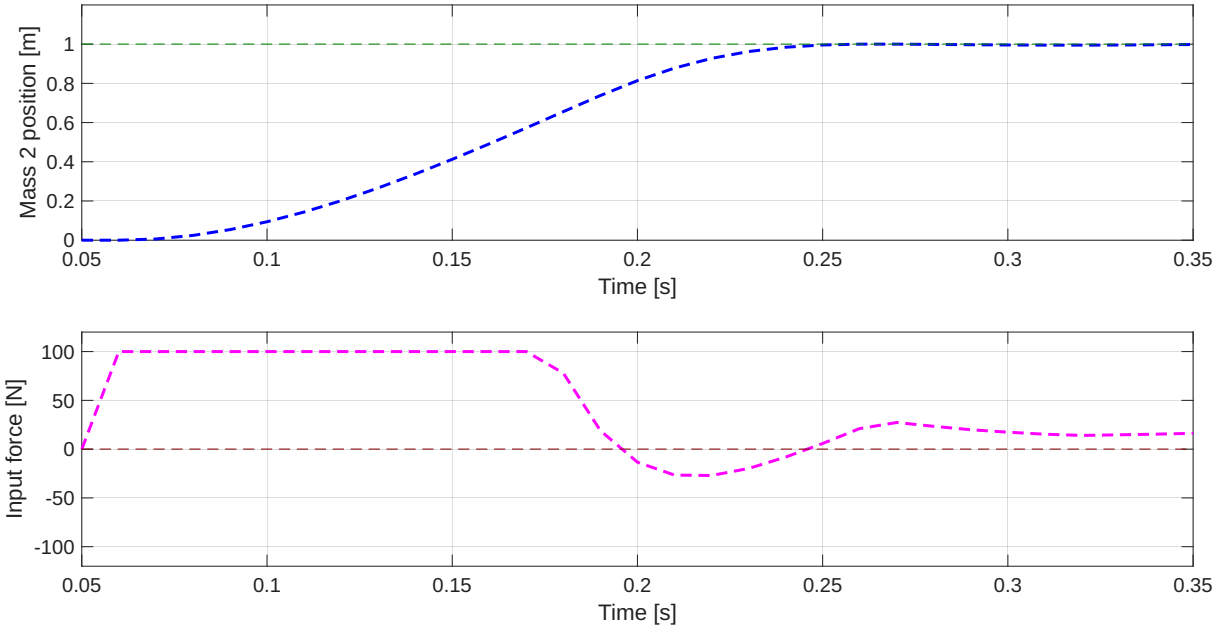


Figure 5.5: Position of mass 2 and force input with DeePC - y limit

Tuning the paramters gave us this result, which can be considered relatively good, we got a short settling time with no overshoot. For these simulations we used MATLAB with YALMIP toolbox and MOSEK solver.

In simulation where systems are completely linear and there is zero noise we get a good result, however, when we consider realistic systems with noise and nonlinearites this DeePC performs poorly - it is not robust. To improve on robustness we implement regularizations to our optimization problem.

5.4 Regularized DeePC

The paper [todo: deepc] introduced robustifications to the DeePC algorithm (5.2) to make it applicable to realistic systems. We will lay out their result and simulate it. The Fundamental lemma is an result which completely falls apart when we introduce measurement noise (or nonlinearity) to our system since there is a high probability that the image of Hankel matrix of input-output data spans the whole space instead of the aforementioned subspace (the behaviour). There is an approach where we can maybe remove the noise from recorded data with SVD or similar techniques, that is good for recorded data but for DeePC where we use real-time data it is not that simple. Now we will formulate regularized DeePC optimization problem:

$$\begin{aligned} \min_{g,y,u,\sigma_y} \quad & \sum_{j=0}^{L_{ref}-1} \left(\|\bar{y}_j - r_{t+j}\|_Q^2 + \|\bar{u}_j\|_R^2 \right) \\ & + \lambda_g \|g\|_2^2 \ + \lambda_y \|\sigma_y\|_2^2 \\ \text{subject to:} \quad & \begin{bmatrix} U_p \\ Y_P \\ U_f \\ Y_f \end{bmatrix} g = \begin{bmatrix} \bar{u}_{ini} \\ \bar{y}_{ini} + \sigma_y \\ \bar{u} \\ \bar{y} \end{bmatrix} \\ & \bar{y}_j \in \mathcal{Y}, \quad j = 0, \dots, L_{ref} - 1; \\ & \bar{u}_j \in \mathcal{U}, \quad j = 0, \dots, L_{ref} - 1; \end{aligned} \tag{5.3}$$

The hard equality constraint in the original problem can cause problems when noise is added, so we impose a soft constraint with slack variable $\sigma_y \in \mathbb{R}^{pL_{ini}}$ to ensure the feasibility of the constraint. We penalize the 2-norm squared of the slack variable with the weight $\lambda_y \in \mathbb{R}$, choosing sufficiently large λ_y "ensures" that $\sigma_y \neq 0$ only if the constraint is infeasible i.e. that data is inconsistent. Two-norm squared regularization on g with weight $\lambda_g \in \mathbb{R}$ is motivated by previous robust optimization problems, more detail in [ref]. In the original paper [ref] 1-norm regularization was used, but in a more recent paper [ref] it was updated to 2-norm regularization.

We test the regularized DeePC on 3.1. Since this example requires a non-zero input for a non-zero steady state we replace $\|\bar{u}_j\|_R^2$ with $\|\bar{u}_j - u_s\|_R^2$ in (5.3), where $u_s \in \mathbb{R}^m$ is the steady state input. We introduce Gaussian measurement noise to the system model like this:

$$y_k = Cx_k + Du_k + v_k, \ v_k \sim \mathcal{N}(0, \sigma^2) \tag{5.4}$$

For this example we took $\sigma = 0.01m$. Secondly, we calculate the steady state input from the model by solving for u_s:

$$\begin{aligned} (I - A_d)x_s - B_d u_s = 0 \\ Cx_s + Du_s = y_s \end{aligned} \tag{5.5}$$

Where y_s is the steady state output, for our example where $y_s = 1\ m$ solving (5.5) yields $u_s \approx 71.94\ N$. This steady state input is, generally, not necessary for DeePC and it can be calculated from data. Now we run regularized DeePC to track the steady state.

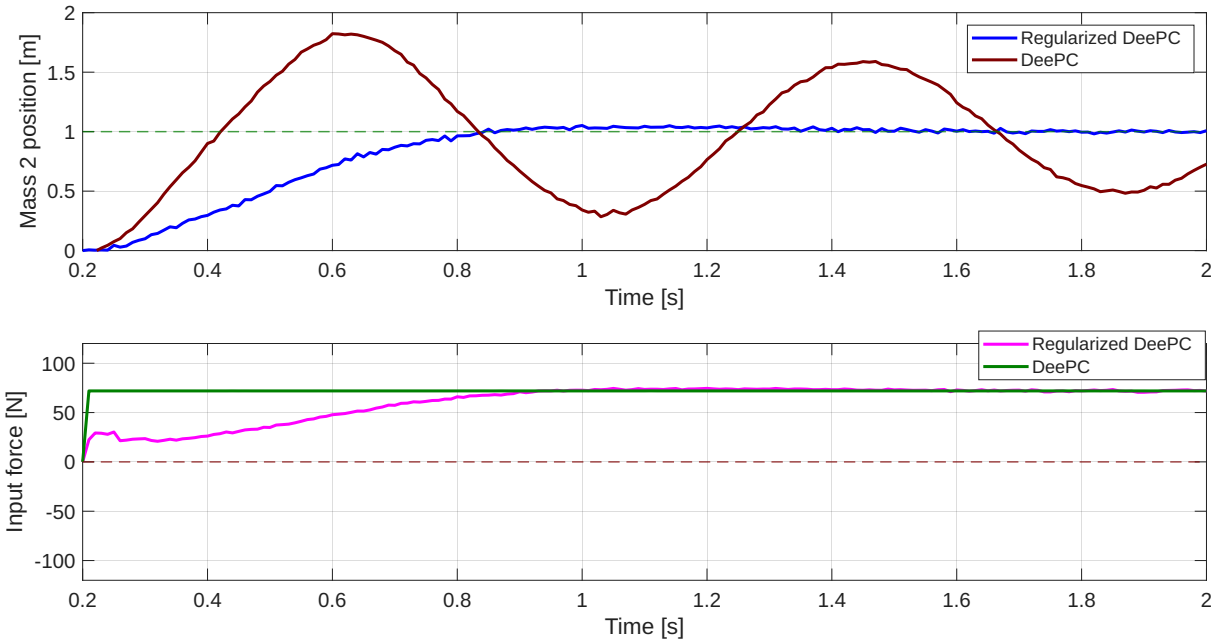


Figure 5.6: Regularized DeePC vs. DeePC

Tuning the parameters of (5.3) gave us a really good result for regularized DeePC and also we can see that DeePC without regularizations performs poorly with the same parameters and data.

6 Conclusion

A Additional Material

Bibliography

[1] Author Name. *Title of the Book or Article*. Publisher or Journal, year.